

令和 7 年度 卒業論文
自律移動ロボットのための価値反復と
A*探索を組み合わせた大域経路計画

中村啓太郎
China Institute of Technology

2026 年 1 月 29 日

謝辞

本研究は、JSPS 科研費 JP24K15127 の助成を受けたものです。

本稿は、千葉工業大学先進工学部未来ロボティクス学科 自律ロボット研究室（上田隆一研究室）において執筆しました。

研究室では、多くのメンバーと共に過ごし、様々な刺激をいただきました。その中で、私の研究を手伝っていただいた皆様に感謝します。

修士2年の吉越さんには、実機実験で使用するロボットの不調の折に大変お世話になりました。長年の経験からアドバイスをいただき感謝します。

また、修士1年の佐々木さんには、ロボットの改修でお世話になりました。ハードウェアの知識の足りない私にとってよい学びとなりました。

修士1年の永木さんには、同じく価値反復を用いた研究を行っていることから、作成したシミュレータ環境を使わせていただき感謝します。

修士2年の船井さん、修士1年の茂さんと川原さんとポルソンさん、同級生である市東さん、鷲尾さん、鈴木さん、藤野さんには、よく研究室での話し相手になっていただき感謝します。私がよく研究に行き詰まると、勝手に話しかけていましたが、快くお相手してください幾度も助けられました。

学部3年の、辻さん、水牧さん、和田さん、根本さんは、私のくだらない話にも付き合ってください感謝します。来年度の活躍を楽しみにしています。

最後に、本研究に取り組み、2度の学会の予稿と本稿を執筆するにあたり、ご指導を頂いた上田隆一教授に感謝します。

目次

謝辞	iii
第 1 章 序論	1
1.1 研究背景	1
1.2 従来研究	8
1.3 研究目的	15
1.4 論文の構成	15
第 2 章 移動ロボットの経路計画問題	17
2.1 問題定義	17
2.2 価値反復アルゴリズムによる経路計画	19
第 3 章 提案手法	23
3.1 価値反復と 2D A*を組み合わせた大域計画	23
3.2 価値反復と 3D A*を組み合わせた大域計画	24
第 4 章 実装	27
4.1 A*の実装と価値反復への適用	27
第 5 章 シミュレータ実験	29
5.1 実験条件	29
5.2 実験結果	32
第 6 章 結論	35
付録 A pure pursuit による経路追従	37
A.1 概要	37
A.2 導出	37
A.3 実装	38
参考文献	41

第 1 章

序論

1.1 研究背景

1.1.1 ロボティクスに対する社会的要請

少子高齢化と労働生産性の課題

21 世紀に入り，先進諸国において少子高齢化に伴う生産年齢人口（15～64 歳の人口）の減少が深刻な社会問題となっている．特に日本においては，国立社会保障・人口問題研究所の推計によると，生産年齢人口は 1995 年をピークに減少傾向にあり，2070 年には約 4,500 万人（2020 年比で約 4 割減）まで落ち込むことが予測されている [国立 23]．この人口構造の変化は，経済成長の鈍化のみならず，社会インフラの維持そのものを脅かす要因となっている．

物流業界においては，物流クライシスと呼ばれる状況が顕在化している．電子商取引（E-commerce）の爆発的な普及により，小口配送の需要が急増している．国土交通省の調査によれば，宅配便取扱個数は年間 50 億個（2023 年度）を超え，過去 10 年間で約 1.3 倍に増加している [国土 24]．トラックドライバーや倉庫内作業員の不足は慢性化しており，労働環境の悪化と配送網の維持困難が懸念されている．また，製造業の現場においても，熟練工の引退に伴う技術継承の問題や，単純搬送作業への人員配置の困難さが指摘されている．

さらに，医療・介護の分野では，高齢者人口の増加に対し，介護従事者の数は圧倒的に不足している．厚生労働省の推計では，2040 年度には約 272 万人の介護職員が必要とされているが，現状のままでは数十万人規模の供給不足に陥る可能性が指摘されている [厚生 24]．病院内での検体搬送，リネン類の回収，あるいは介護施設における見守りや配膳など，定型的な業務の負担軽減は，強い需要がある．

従来のロボットによる業務自動化と限界

こうした労働力不足を補う手段として，特に工場内物流において，無人搬送車（automated guided vehicle: AGV）の導入が進められてきた．AGV は，床面に敷設

された磁気テープや反射テープ，あるいは2次元コードといった物理的なガイドをセンサで読み取りながら走行するロボットである．これにより，従来人の行っていた台車を押すような運搬作業をロボットが代替することが可能になった．AGVは環境が固定されており，かつタスクが定型的である場合に高い効率と信頼性を発揮する．

一方で，AGVの導入と運用には，物理的なガイドを必要とするという制約から以下のような構造的な限界が存在すると指摘されている [Fragapane 21]．これらの限界は，人や有人フォークリフトが頻繁に行き交う物流倉庫や，一般の人々が存在する病院・商業施設といった動的な障害物の多い環境へのロボット導入を阻む大きな障壁となっていた．

1. インフラ敷設コスト: AGVに走行させたい経路のすべてにガイドを設置する必要があり，導入時の工事コストや期間が甚大である．また，AGVだけでなく，このガイドのメンテナンスも必要となる．
2. レイアウト変更の柔軟性欠如: 製造ラインや倉庫のレイアウトなどによりAGVの通る経路を変更する際，ガイドの敷設し直しが必要となり，多大なコストとダウンタイムが発生する．
3. 動的環境への非適応: 想定された経路上に障害物が存在した場合，AGVはその場で停止することしかできず，回避して目的地へ向かうことができない．また，人がセンサを遮り，ロボットが自分自身の位置がわからなくなるなど，

AGVから自律移動ロボット (autonomous mobile robot: AMR) へ

以上のAGVの課題を克服するため研究，開発が進められてきたのが，自律移動ロボット (AMR) である．AMRは，図1.1に示すように，物理的なガイドを必要とせず，LiDAR (light detection and ranging) やカメラといった外界センサを通じて周囲の環境の情報を得る．このセンサ情報と事前に作成した環境地図を照らし合わせることで地図内の自身の位置を推定し，物理的なガイドを必要とせず走行する．

AMRは，AGVと違い，経路上に障害物を検知すると，回避経路を生成し，その経路を追従することでタスクを継続することが可能である．これは，AMRが走行する経路は，物理的なガイドによる経路ではなく，ソフトウェア上の経路であるために，動的に経路を変更することが可能である．

また，物理的なガイドを必要としないため，導入時の工事が不要であり，ソフトウェア上の設定変更のみで走行エリアや経路を変更できる．これらの特性により，AMRは従来AGVが導入困難であった動的な環境への適用が進んでおり，Society 5.0の中核を担う技術として期待されている [内閣 16]．

さらに，AMRは，決められた経路を巡回するタスクだけではなく，ユーザーの指定する任意の目的地へ向かうタスクを遂行することも可能である．ソフトウェア上の経路を計算によって求める技術が発展し，動的に，実用的な時間（例えば， $< 1\text{ s}$ ）で，経路を計算することができる．複数の目的地を順々にロボットへ与えることで，経路を巡回するタスクも任意の目的地へ向かうタスクとして，可能となる．

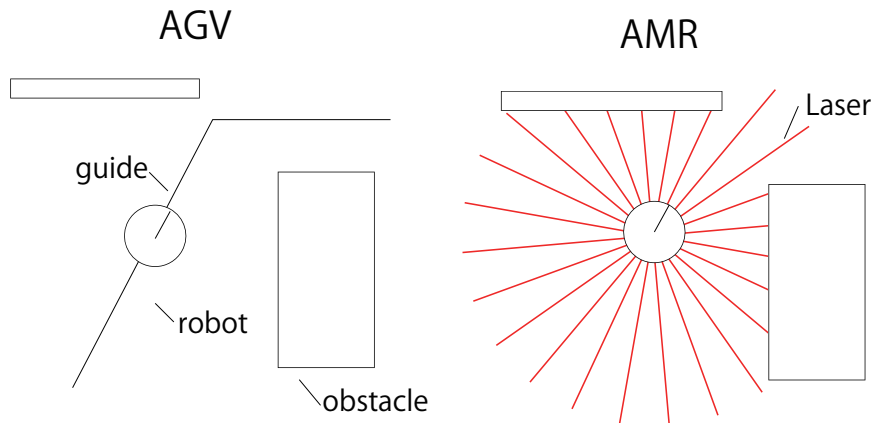


図 1.1 Comparison of AGV and AMR

自律移動ロボットの屋外への適用

工場や倉庫といった屋内環境で培われた AMR の技術は、近年、より複雑かつ広範な屋外環境へとその適用範囲を拡大している。特に、労働力不足が深刻な物流や農業分野において、屋外対応型 AMR の実用化が急速に進展している。

物流分野では、配送拠点からエンドユーザーへの最終区間であるラストワンマイルの配送コスト削減が課題となっている。ライトワンマイルの配送は、配送先が多く、多くの人手を要している。多くの人手を要している。配送時間の短縮と環境負荷の低減を実現する有効な手段として位置付けられている [Alverhed 24]。日本国内においても、2023 年 4 月に施行された改正道路交通法により、自律移動ロボットが遠隔操作型小型車として定義され、届出制による歩道走行が可能となった [日本 22]。これにより、パナソニックや楽天といった企業が図 (1.2) に示すようなロボットを用いて、住宅街での配送実証を行っており、社会実装が進みつつある。

また、農業分野では、農林水産省が人手のかからないスマート農業を推進している [日本 24]。実際に、クボタやヤンマーなどの農機メーカーが有人監視下での自動走行（レベル 2）および無人自動走行（レベル 3）に対応したロボットトラクターを市場投入している [株式 17, 横山 19]。

しかしながら、屋内環境と比較して、屋外環境はロボットにとってタスクの遂行が困難となる要素が多い。第一に、ロボットの走る路面のロボットへ与える影響がある。多くの移動ロボットは車輪型 [原 25] であり、平らで傾きのない屋内では、路面からロボットへ与える影響は小さい。一方、屋外では、路面が必ずしも水平でなく、段差や凹凸もあり、ロボットの姿勢の急激な変化を招いたり、振動によってセンサ情報にノイズを与えたりといった影響がある。第二に、天候や季節による路面状況の変化や、時間変化による照明条件の変化がある。雨、泥、雪、あるいは落ち葉などは、風景の見た目や形を大きく変化さ



図 1.2 Rakuten's Autonomous Delivery Robot(Source: [楽天 25])

せる。また、直射日光による白飛びや逆光、夜間の低照度、朝日、夕日による色温度の変化といった光環境の変動は、視覚情報の大きな変化を伴う。

これらの環境要因からロボットがどのように周囲を認識し、どのように自身の位置を知り、どのように経路を引くかという課題が発生する。次項では、自律移動を実現するために現代の AMR がどのような技術要素によって構成されているか、どのような課題が存在するか、そのシステム概要について述べる。

1.1.2 自律移動ロボットの技術構成

センシング

センシングは、ロボットが周囲の環境から情報を得るための技術である。以下に、移動ロボットで用いられる代表的なセンサ技術を示す。

- **LiDAR** レーザー光を照射し、反射光が戻ってくるまでの時間 (Time of Flight) や位相差から距離を計測するセンサ。北陽に代表される 2 次元平面をスキャンする 2D LiDAR が主流であったが、図 1.3 左に示す Velodyne Lidar に代表される 3 次元点群を取得可能な 3D LiDAR が主流になりつつある。近年では、図 1.3 右に示す Livox など中国メーカーの廉価な LiDAR が多く流通している。
- **カメラ**: 可視光線を計測し、人の知覚する色の分布を捉えることができるセンサ。RGB 画像に加え、深度情報を取得できる RGB-D カメラやステレオカメラが利用される。Visual SLAM (後述) や物体認識との親和性が高い。
- **オドメトリ (odometry)**: 車輪の速度や回転数 (エンコード値) や IMU (慣性計測装置, *inertial measurement unit*) のデータから、ロボットの相対的な移動量を

推定する技術 [前山 97]. 短期的には高精度だが, 累積誤差が生じるため単独では長距離移動に適さない. エンコーダや IMU はロボットの周囲の情報ではなく, ロボットの内部の情報を取得するため, 内界センサと呼ばれる.

- **GNSS (global navigation satellite system)**: アメリカの GPS (Global Positioning System) や日本の QZSS (Quasi-Zwynth Satellito Ststem, みちびき) といった衛星測位システムの総称. 屋外環境において地球上での位置を取得する主要な手段となる [塚越 16]. 基準局を用いて誤差の補正を行う RTK-GNSS (real time kinematic-GNSS) により, 数センチメートルの精度での測位が可能となり, 配送や農業を行うロボットに広く採用されている.
- **無線通信 (WiFi/5G)**: アクセスポイントからの信号強度や電波の到達時間を用いて測位を行う.

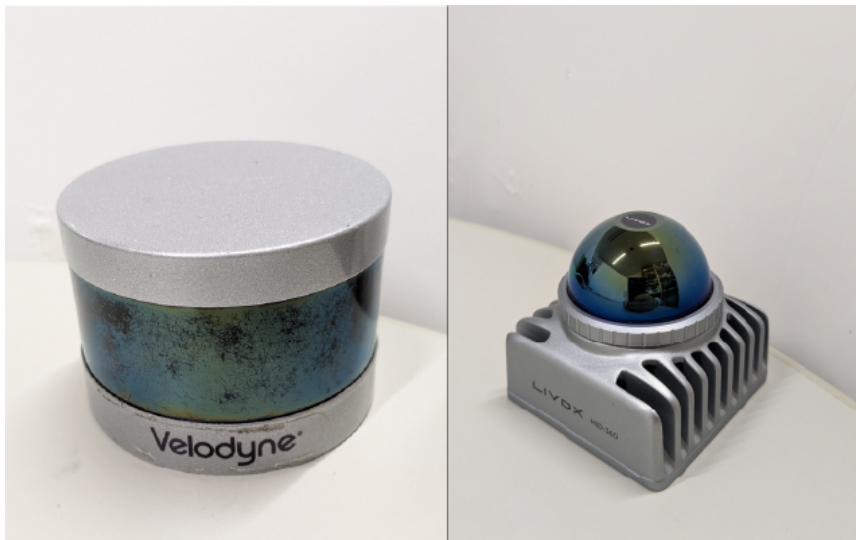


図 1.3 3D LiDAR (Left: Velodyne, Right: Livox)

自己位置推定 (localization)

自己位置推定は, 周囲の環境を測定するセンサの情報からロボットがロボット自身の位置を推定する技術である. 単に, 自己位置推定といった場合は, 各地点で環境から取得できる情報を記した地図をあらかじめロボットが持っており, その地図上の自身の座標を推定することを指す. 地図を持たない場合は, SLAM (simultaneous localization and mapping) とよばれる.

■ **SLAM** SLAM はロボットがセンサ情報と制御入力だけで地図を作る技術である [Thrun 05]. SLAM は, 主に用いるセンサごとに LiDAR SLAM と Visual SLAM に大別される. それぞれの SLAM ごとに, 代表的な実装例を以下に示す [友納 20].

- **LiDAR SLAM**: 2D LiDAR を用いたものでは, パーティクルフィルタベースの

GMapping[Gerkey 09] やグラフベースの Cartographer[Kohler 16] などがある。3D LiDAR では、スキャンマッチングによる Velodyne SLAM[Moosmann 11] や LOAM[Zhang 14] などがある。近年の日本では、GLIM[Koide 24] がシェアを伸ばしている [原 25]。

- **Visual SLAM:** カメラ画像の特徴点を用いる ORB-SLAM[MA 15] や、直接法と呼ばれる密な点群を用いる LSD-SLAM[Engel 14] などがある。

■**地図あり自己位置推定 (map based localization: MBL)** 地図座標系上での位置を推定する技術は、オドメトリの累積誤差を補正する方針で、様々な手法が開発されてきた。1990 年代以降、Thrun らによって提唱された確率ロボティクス (Probabilistic Robotics) [Thrun 05] は、センサのノイズや環境の不確実性を確率分布として扱うことで、ロバストな自己位置推定を可能にした。代表的な手法として、(拡張) カルマンフィルタとパーティクルフィルタ (particle filter) がある。

パーティクルフィルタの実装方法として広く使われているのが、モンテカルロ位置推定 (Monte Carlo localization: MCL) [Fox 99] である。これは、ロボットの存在確率分布を多数の粒子 (パーティクル) で表現し、センサ観測と動作モデルに基づいて粒子の重みと位置を更新する手法である。MCL は、開発された当時、ロバスト性において他の手法より優れており、現在の AMR のデファクトスタンダードとなっている。

自己位置推定の成熟により、ロボットは環境内での自身の位置を推定できるようになった。また、SLAM により、障害物の配置や通行可能な領域を表す環境の地図を作成することも可能となった。

経路計画 (path planning)

ロボットが実際にタスクを遂行するためには、自己位置推定だけでなく、現在地 (start) から目的地 (goal) まで、障害物を回避しつつ、かつ効率的 (最短時間, 最小エネルギー など) に到達するための行動を決定しなければならない。この行動を決定するための技術を経路計画という。

経路計画は、自己位置推定の結果と環境地図、目的地を入力とし、ロボットのアクチュエータ (モータ) への制御指令を出力とする、自己位置推定がいかに正確なものであっても、経路計画が不適切であれば、ロボットは遠回りをするか、狭い通路で立ち往生するか、最悪の場合は障害物と衝突する危険性がある。

前節で述べたように、社会実装が進むにつれて、ロボットが稼働する環境はより複雑化している。静的な障害物だけでなく、人や他のロボットといった動的な障害物、路面に存在する微小な障害物によるノイズや、環境の変化によるセンサ計測の不確実性などがその例として挙げられる。これらの要因を考慮し、安全かつ条件に対して最適な経路をリアルタイムに導出することは、自律移動ロボットの社会実装を進めるにあたって、重要な研究課題である。次節では、この経路計画問題に関する従来研究を概観し、本研究で扱う価値

反復の位置付けを明らかにする.

1.2 従来研究

移動ロボットの経路計画は、動的計画法（dynamic programming）[Bellman 57] に該当し、ロボットの位置と移動の表現に関して決定論的なアプローチと、確率的なアプローチに大別できる。決定論的なアプローチでは、ロボットの位置は、座標で表され、移動は、制御司令によって次の状態（位置と角度）に必ず移動すると仮定する。これにより、経路を求めるのにかかる計算量を削減することができる。

一方で、確率的なアプローチでは、ロボットの位置を確率分布で表し、移動は、分布を動かすと表現することで、外乱の影響によって、必ずしも次の状態に移動できないかもしれないという不確実性を計算に組み込むことができる。計算量は決定論的なアプローチに比べ増加することが多いが、より安全な経路計画を行うことができる。次節からそれぞれのアプローチの代表的な手法について述べる。

1.2.1 決定論的アプローチによる経路計画

探索手法を用いた経路計画器

経路計画には、離散数学におけるグラフ探索問題を応用して経路を求める手法がある。グラフ探索問題は、与えられたノードとノードを繋ぐエッジからなるグラフ上に、スタートとゴールのノードが設定され、エッジをたどりノードを移動してゆき、最短の移動で、ゴールのノードにたどり着く 1 通りのノードとエッジの列を求める問題である。経路計画に適用するとき、グラフは、環境の各要素を表すノードと、そのノード同士の関係を表したエッジから構築する。これにより、現在地（スタート）からゴールまでの経路を探すことをグラフ上を探索する問題に帰着できる。

グラフの構築方法には、環境の地図を格子状（グリッド）に分割し、各セルをノード、隣接セルへの移動をエッジとしてモデル化するグリッドベースの手法と、空間内にノードをランダムにサンプリングしてグラフを構築し、探索を行う手法がある。後述する Dijkstra 法 [E.W. 59] や A* アルゴリズム [Hart 68] は、主にグリッドベースの手法のグラフで探索を行う。

グリッドベースの手法では、多くの場合、セルに、障害物があり通行不可能、障害物がなく通行可能、不明の 3 種類を設定し、障害物がなく通行可能なセルからなるノードだけをたどる経路を算出することが求められる。また、エッジは、1 つのセルから隣接する 8 つのセルにエッジが繋がれており、繋ぐノード間の距離を重みとして持つ。軸方向の距離（重み）を 1 としたとき、斜めに移動するエッジは、その $\sqrt{2}$ 倍になる。

ただ、グリッドベースの手法は、格子状に区切ったすべてのセルをノードとするため、広大な地図や、解像度の高い地図においては、グラフに含まれるノードの数が多くなり、探索にかかる計算量が多くなる問題点がある。これをランダムにサンプリングしたノードで構築したグラフに置き換えることで、適切な量のノードをサンプリングできれば、グ

リッドベースのグラフより少ないノードの数で十分に空間を覆ったグラフを構築することができる。代表的な手法に RRT（後述）がある。

■Dijkstra 法 Edsger W. Dijkstra によって考案された Dijkstra 法は、非負の重み付きグラフにおける単一始点最短経路問題を解くアルゴリズムである [E.W. 59]。経路計画では、各エッジの重み（＝距離）は、常に正であり、この重みを移動にかかるコストとして、スタートのノード n_s からゴールのノード n_g までこのコストが最小になるような、ノードの列（＝経路）を算出するため、Dijkstra 法を応用し問題を解くことができる。Dijkstra 法では、 n_s からあるノード n まで、累加したコスト（＝移動する距離）を $g(n)$ と書き、実コストとよぶ。 n_s から移動可能な隣接するノードへの $g(n)$ を計算し、計算したノードの中から $g(n)$ が最小のノードのコストを確定させる。次に、新しく確定したノードを含むすべての確定したノードから移動可能な隣接するノードの $g(n)$ を再度計算し、その中から最小のものをを選び、コストを確定させる。この手順をゴールのコストが確定されるまで続け、探索範囲を広げていく。

常に $g(n)$ が最小となるノードから移動可能なノードからコストを計算することで、数学的に最短経路が保証される。しかし、図 1.4 に示すように、探索が全方位に均等に広がるため、ゴールの方角情報利用されず、探索範囲が膨大になる欠点がある。

■A*アルゴリズム A* (A-Star) アルゴリズムは、Dijkstra 法にヒューリスティック関数 $h(n)$ を導入することで探索を効率化した手法である。[Hart 68] Dijkstra 法の $g(n)$ に替わり用いる評価関数 $f(n)$ を

$$f(n) = g(n) + h(n) \quad (1.1)$$

のように定義する。ここで、 $g(n)$ は、 n_s からノード n までの実コスト、 $h(n)$ はノード n から n_g までの推定コストである。 $h(n)$ は、ヒューリスティック関数と呼ばれ、人の手によって設計される。移動ロボットの場合は、 $h(n)$ として現在のセルからゴールセルまでのユークリッド距離やマンハッタン距離が用いられる。 $h(n)$ が実際の最短コストを決して上回らない（許容的な、admissible）場合、A*アルゴリズムは最適解を保証しつつ、Dijkstra 法よりも少ない計算量で解に到達できることが多い。現在でも最も広く使われている標準的なアルゴリズムである。

■RRT (rapidly-exploring random tree) RRT は、グラフをツリー状として、ランダムにサンプリングされた点に向かってツリーを拡張していくことで、経路を探索する [LaValle 98]。計算量が特に少ないという利点がある一方で、RRT によって生成される経路は最適性が保証されず、図 1.5 に示すように、ジグザグで遠回りな経路になりがちである。これを改良した RRT*[Karaman 11] は、無限回の探索で最適な経路が見つかる保証を有するが、計算量は、RRT に比べ多いものとなる。

これらの、スタートからゴールまでの経路を求めることを、大域経路計画（global path planning）と呼び、ロボットは算出した経路を追従することで、ゴールに向かうことがで

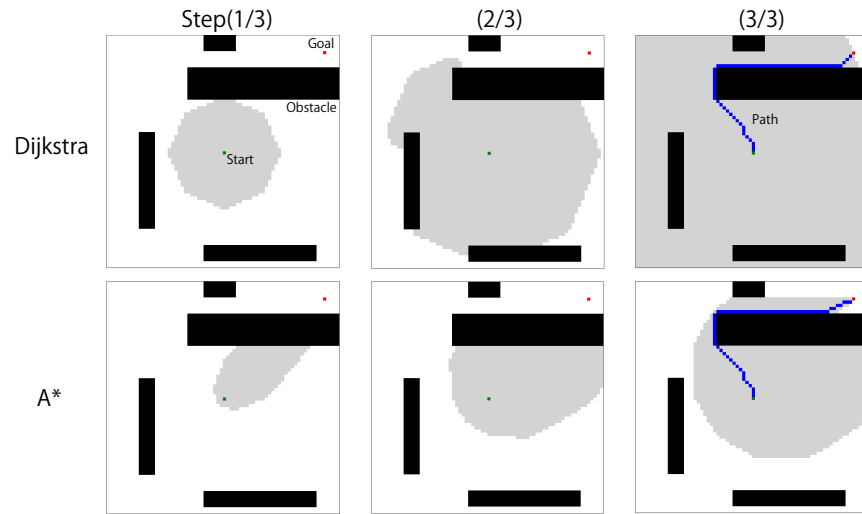


図 1.4 Comparison of search ranges between Dijkstra and A*
(gray: searched, white: unsearched)

きる。しかし、この経路は、あらかじめ地図に記された静的な障害物のみを回避する経路であり、動的な環境において現れる経路上の障害物を回避することが求められる。動的な障害物の回避方法として、動的な障害物を地図に書き加え、もう一度探索を行う方法がある。これをリプランニングと呼び、再度探索を行うタイミングは、数秒～数分に 1 回か、または通路が塞がれるなどトポロジカルな構造の変更時などに設定される。

障害物を回避する他の方法として、動的な障害物を避ける短い経路を算出しながら大域経路を追従する局所経路計画 (local path planning) がある。局所経路計画は、大域経路に追従しつつ、搭載されたセンサで検知した未知の障害物や動的障害物をリアルタイムに回避するための制御入力を生成する。更新頻度は高く (10Hz～100Hz)、ロボットの運動学的制約や動力学的制約を考慮する。代表的な手法には、dynamic window approach (DWA) や model predictive control (MPC) などがある。

障害物回避のため、リプランニングや局所経路計画を併用する方法を用いると、ロボットの制御が振動するチャタリングが発生し、移動が困難になる場合がある。チャタリングとは、生成される経路が短時間に幾度も切り替わることで、制御司令が激しく変化する現象である。チャタリングは、リプランニングにおいては経路が頻繁に更新されることによって、局所経路計画器を併用においては大域経路と局所経路の競合によって引き起こされる。

経路計画器は、チャタリングを回避するか発生しても復帰できるように設計する必要がある。チャタリングに陥ると、進行方向が何度も切り替わるなどしてロボットがスタック (立ち往生) してしまう可能性がある。チャタリングの回避は、探索手法を用いた経路計画器でも適切なパラメータやアルゴリズムの選定によって可能である。

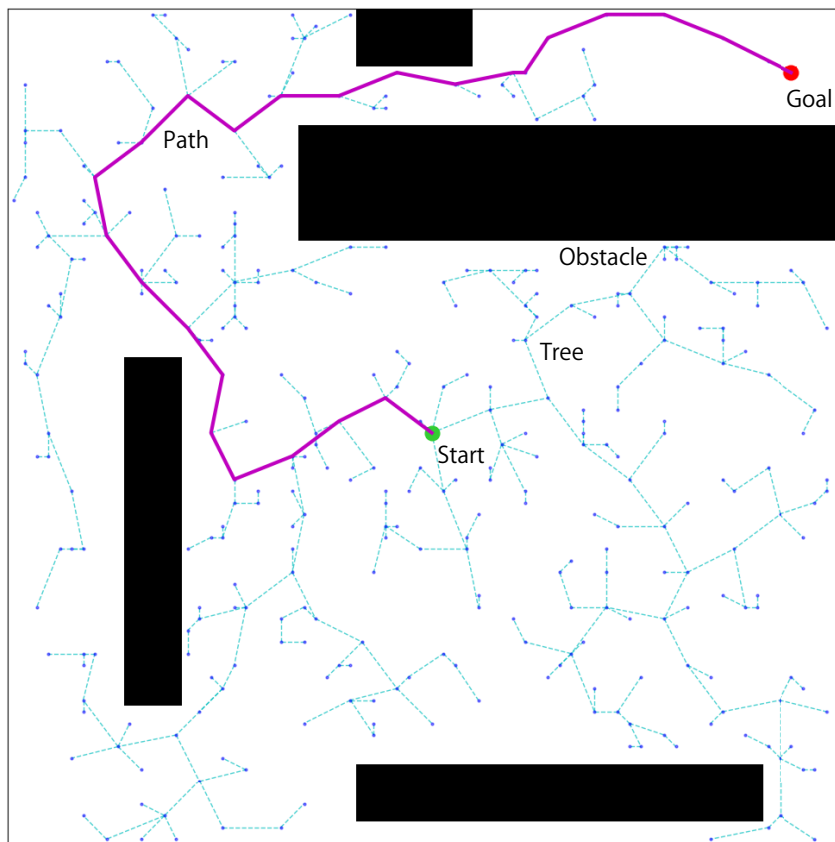


図 1.5 RRT search result

ポテンシャル法

このチャタリングを回避する方法の1つとして、グラフ探索手法が、経路という点の集合を求めるのに対し、環境全体に対して値（ポテンシャル）を対応付けたポテンシャル関数を定義し、値の勾配にしたがって移動するポテンシャル法がある。大域経路計画と局所経路計画を同一の計算方法で計算でき、両者の矛盾（＝チャタリング）を回避できる利点がある。他にも、自己位置推定の結果のジャンプや振動に対応しやすいことも利点として挙げられる。

■人工ポテンシャル法 (artificial potential fields) 人工ポテンシャル法は、ゴールからの引力と障害物からの斥力を合成したポテンシャル場を構築する手法である [Khatib 85]. 電磁気学のクーロン法則を応用し、仮想的に、ロボットを弱い正の電荷を持つ粒子とし、障害物には強い正の電荷、ゴールには強い負の電荷を与える。すると、ロボットはポテンシャルの勾配にしたがって最もエネルギーが低い方向へ移動するだけで障害物を回避し、ゴールへ向かうことができる。これは、動的な障害物を追加しても計算負荷が非常に軽く、リアルタイムな障害物回避に適している。

しかし、人工ポテンシャル法には、ゴール以外の窪みにハマって出られなくなる現象で

ある局所解 (local minima) に陥ってしまうという問題がある。U 字型の障害物などに遭遇した場合、引力と斥力が釣り合ってしまう、ゴールに到達する前にポテンシャルの極小値で停止してしまう現象が発生する。この問題を解決する手法は、多く提案されているが、理論的な枠組みから解決に取り組む手法がある。

■ナビゲーション関数 (navigation functions) Koditschek と Rimon[Koditschek 90] が提唱したナビゲーション関数は、幾何学的な構成空間において、ゴールのみを唯一の大域的最低点 (global minimum) とし、その他すべてのポテンシャルの勾配が 0 となる点が周辺にポテンシャルの下る勾配を持つような不安定な鞍点 (saddle point) となるように設計された特殊なポテンシャル関数である。ナビゲーション関数が構築できれば、その勾配に従うだけで、ロボットはどのような初期位置からでも必ずゴールへ到達できることが数学的に保証される。

グリッドマップ上におけるナビゲーション関数の実装例として、勾配法 (gradient method) [Konolige 00] が挙げられる。勾配法は、任意のセルを p_0 として、ゴールを p_k としたとき、2 点間を繋ぐ k 個の連続したセルの経路 $P_k = \{p_0, p_1, \dots, p_k\}$ に対して、ナビゲーション関数 $N(p_0) \in \mathbb{R}$ を

$$N(p_0) = \min_{P_k} \left\{ \sum_{i=0}^k I(p_i) + \sum_{i=0}^{k-1} D(p_i, p_{i+1}) \right\} \quad (1.2)$$

にしたがって計算する。ここで、 $I(p_i)$ は、 p_i の路面の凹凸や、障害物との距離など、そのセルを通るとき、ロボットにかかる悪影響を数値化したコスト、 $D(p_i, p_{i+1})$ は、2 点 p_i, p_{i+1} のユークリッド距離である。

この N を wavefront アルゴリズム [Arkin 90] を改良した LPN というアルゴリズムで計算し、これをポテンシャル関数とする。こうして得られたポテンシャル関数は局所解を持たず、大域的に最適な経路情報を内包している。

1.2.2 確率的なアプローチによる経路計画

以上の決定論的な大域経路計画に対し、本研究で取り扱う価値反復 (value iteration) は、移動の不確実性を確率的な状態遷移として扱えるようにしたマルコフ決定過程を解くアルゴリズムである。これにより、環境の不確実性を確率的に扱うことにより、外乱に対してロバストな方策を得ることができる。

価値反復は、計算量が多いという問題が知られているが、上田らは、将来的には計算機の性能が向上し、この計算コストの問題が解決されるとして現在の移動ロボットで使われるミドルウェア (ROS) 上で価値反復 ROS パッケージとして実装した [Ueda 23, 上田 21, 上田 24]。文献 [Ueda 23] では、実機上でリアルタイムに経路計画を行えることが報告されている。

価値反復では、環境をマルコフ決定過程 (MDP) としてモデル化する。MDP は、状態空間 S 、行動の集合 A 、状態遷移モデル P 、報酬モデル R の 4 つ組で定義される

[中居 22, 上田 19]. MDP では、次の状態は、その 1 つ前の状態によって決まり、1 つ以前の状態は、影響しないというマルコフ性と、完全に状態が観測可能であるという 2 つの仮定を置いている．決定論的なグラフ探索とは異なり、MDP では、「行動 a を行った結果、確率的に状態 s' へ遷移する」という状態遷移の不確実性を考慮して経路計画を行うことができる．

価値反復 ROS パッケージは、MDP を近似した有限マルコフ決定過程 (finite MDP) を解く [上田 19]. finite MDP では、 $\mathcal{A}, \mathcal{S}$ が離散化され、有限の数の要素を持つ集合となる．解くというのは、方策 (policy)

$$\pi : \mathcal{S} \rightarrow \mathcal{A} \quad (1.3)$$

を求めることである．また、 π を計算するために、状態に対して値を対応付けた状態価値関数 (state value function)

$$V : \mathcal{S} \rightarrow \mathbb{R} \quad (1.4)$$

を計算する．この V は、ポテンシャル関数やナビゲーション関数と同等なものである．

具体的には、 π を用いた際の V を

$$V(s) \leftarrow \sum_{s' \in \mathcal{S}} P(s'|a, s') [R(s, a, s') + V(s')] \quad (1.5)$$

と計算する． V は、目的地に含まれる $s \in \mathcal{S}$ に対して 0、それ以外の s に対しては、無限大 (実装上は大きな定数) に初期化されており、この $V(s)$ を最小化するような最適な方策 $\pi^*(s) \in \mathcal{A}$ を選ぶことで式 (1.5) は、ベルマン方程式 (Bellman equation)

$$V^*(s) = \min_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s'|a, s') [R(s, a, s') + V^*(s')] \quad (1.6)$$

を満たす．この式 (1.6) を全状態に対して適用し、反復計算することで、 π^* と、環境内のあらゆる場所からゴールへ向かうための最適状態価値関数 V^* (最適なポテンシャル関数) が波及的に計算される．

価値反復は、ナビゲーション関数と同様に、局所解が発生しないという強力な数学的保証がある．また、確率的な状態遷移を用いて V を計算することで、実ロボットの移動誤差を、価値の期待値として計算し、行動を判断できる．具体的には、ロボットが壁際を走行する際のリスクを考慮した経路計画が可能になることを示している [Thrun 05].

しかしながら、価値反復は全状態空間を離散化し、計算を行うため、広大な環境においては計算コストが甚大になり、現代の計算機でも、大域的な経路計画には時間がかかることが分かっている．価値反復 ROS パッケージの状態空間 $\mathcal{S}$ は、グリッドベースの手法よりも角度の次元の分、一つ次元が大きいと、地図が大きくなったり解像度が上がったたりする際の計算コストの増加が Dijkstra 法や A* アルゴリズムよりも大きい．文献 [Ueda 23] の例では、広大な環境を高い解像度でグリッド化すると、状態数は 6 億を超え、15[m] の直線状の経路の計算に 30[s] 程度かかると算出されている．この計算にかか

る時間というのは、目的地から V の勾配がロボットが居る地点まで届くまでにかかる時間と考えることができる。

V の勾配が届くまでロボットが走り出すことができない。 V は、目的地以外は、無限大で初期化されているため、移動開始しばらくは、ロボットの周辺の V は一様である。 π は、 V を最小化する行動を算出するが、 V が一様であると、選ばれる行動は計算上の誤差によってランダムとなる。 V の勾配が届くまでは、有意な行動をロボットは取れないのである。

1.3 研究目的

以上から、価値反復 ROS パッケージにおいて、現代のコンピュータを用いて単純な経路を求める場合でも、走り出すまでに時間がかかる問題があることが分かる。ロボットが走り出すためには、価値反復によって生成される V の勾配が、ロボットの居る地点まで届く必要がある。

そこで、価値反復よりも計算量の少ないアルゴリズムで経路を算出し、その経路を用いることで勾配を生成すると、ロボットがより早いタイミングで走り始めることが期待できる。

よって、本研究の目的を、「価値反復 ROS パッケージを用いたナビゲーションにおいて、走り出しまでの時間を短縮すること」とする。走り始めるまでの時間が短くなることで、移動全体にかかる時間も短くすることができると考えられる。

V は、どのような初期値からも収束するため、 V の値を書き換えは、 V^* に影響は与えないが、計算中の V に対しては、影響を与えることができる。この影響により全体の移動時間が伸びることがないようにしなければならない。

1.4 論文の構成

- 1 章では、移動ロボット研究の背景、従来研究、本研究の目的を述べた。
- 2 章では、移動ロボットの経路計画問題について述べる。
- 3 章では、提案する手法について述べる。
- 4 章では、3 章の手法の実装について述べる。
- 5 章では、3 章のアルゴリズムについて、移動時間の短縮の効果を評価する。
- 6 章でまとめる。

第 2 章

移動ロボットの経路計画問題

2.1 問題定義

2.1.1 環境の定義

本研究では，移動ロボットを平面上で自律移動させる問題を扱う．これは，移動ロボットのプロセス間通信や，プラットフォームとしてよく用いられているミドルウェア ROS 2[Macenski 22] で広く用いられている Navigation2[LLC 18]（ROS 2 の標準ナビゲーションパッケージ）が扱う問題と同様の問題である．

ロボットは，環境を移動し，与えられた目的地の座標まで，障害物との衝突を避けながらできる限り短い時間で到達しなければならない．この目的地は，ロボットが行動を始めるタイミングで与えられる．図 2.1 にその環境の例を示す．

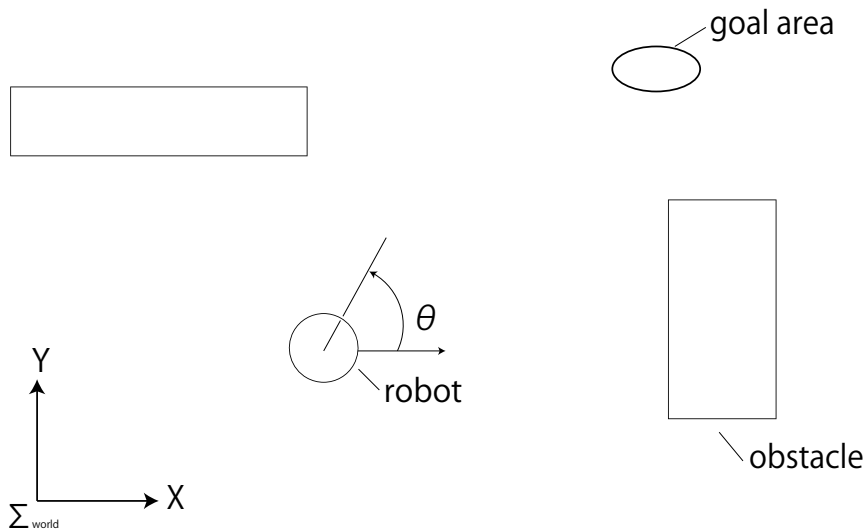


図 2.1 An enviroment for mobile robot navigation

環境には，世界座標系が 2 次元の直交座標系で設定されており，ロボットは，位置 (x, y) と， x 軸となす角 θ を向きとして持っている．これらをまとめてロボットの状態（位置と

向き) $s = (x, y, \theta)$ と, 3つの変数で表す.

目的地(ゴール)は, XY 平面上の座標あるいは, $xy\theta$ 空間上の座標として与えられ, 図中の goal area のようにその座標を中心とした XY 平面上の領域や, $xy\theta$ 空間内の領域として扱われる. 実装によっては, 任意の領域を目的地とすることができる.

環境中には, ロボットの移動を阻む障害物が存在しており, その位置が移動しない, 建物などの固定障害物と位置の変化する未知/移動障害物が存在する. 固定障害物でも, 年や月といった時間スケールでは, 存在の有無が変化したり位置が移動したりするが, 本研究では, 取り扱わない. 未知障害物は, 探索手法を用いる場合, 別途対策が必要となるが, 価値反復 ROS パッケージは既存の機能で対処可能である. 本研究では, 局所経路計画である未知障害物の回避も取り扱わない.

2.1.2 マルコフ決定過程 (Markov decision process: MDP) による定式化

この環境を MDP という枠組みで定式化する. MDP は, エージェント(ここでは, 移動ロボット)が環境と相互作用しながら学習・行動決定を行うための数理モデルであり, 以下の4つの要素の組 $\langle S, A, P, R \rangle$ で定義される. ここで, MDP は, マルコフ性を仮定しており, これは, 時刻 t から $t+1$ の間には, $\langle S, A, P, R \rangle$ 以外のものは, 関与しないという性質である. 例えばロボットを長時間動かすとなど, ロボットの動きが悪くなり P が変化するが, このようなロボットをモデル化する際には, マルコフ性を保つために, S にロボットをメンテナンスしてから動かした時間などを追加する必要がある.

1. 状態空間 S (**state space**): ロボットが取り得るすべての状態の集合. 2.1 節で述べたように, ロボットは, 位置 (x, y) とロボットの方位 θ を持つ. これらを合わせた (x, y, θ) が一つの状態 $s \in S$ に対応する. また, 状態のうち, 目的地に含まれる状態を終端状態と呼び, 終端状態の集合を $S_f \subset S$ とする.
2. 行動の集合 A (**action space**): 各状態でロボットが選択可能な行動の集合. 価値反復 ROS パッケージでは, 速度 $v[\text{m/s}]$ と角速度 $\omega [\text{rad/s}]$ の組で表される. 例えば直進とカーブの行動の組は, $a = (v, \omega)^T = (0.3, 0)^T, (0.2, 0.1)^T \in A$ と表される.
3. 状態遷移モデル $P(s'|a, s)$ (**transition probability**): 状態 s で行動 a を選択したときに, 次の時刻に状態 s' へ遷移する確率.

$$P(s'|a, s) = \Pr\{S_{t+1} = s' | S_t = s, A_t = a\} \quad (2.1)$$

決定論的な環境 (A^* などが想定する世界)では, ある行動を行えば 100% 意図した隣接セルへ移動する. しかし実環境では, 移動はタイヤのスリップや慣性, 段差などといった要因によって誤差が含む. MDP ではこの不確実性を確率分布としてモデル化する. 例えば, 「前進」を選択しても, 10% の確率で左右のセルに移動し, 5% で行き過ぎて奥のセルに移動する, といった表現が可能である.

4. 報酬モデル $R(s, a, s')$ (**reward function**): 状態遷移に伴って得られる評価でス

カで表される。経路計画問題においては、コストの最小化または負の報酬の最大化として定式化される。本稿では、 $R(s, a, s') \geq 0$ と設定し、コストの最小化として定式化する。例えば、移動にかかる時間やエネルギーを表現するため、各ステップごとにわずかなコスト（ステップコスト）を与える。また、路面の凹凸などといった移動へ悪影響を与える要素をもつ状態にはわずかなコストを与え、固定障害物をもつ状態には、大きなコストを与える。

ベルマン方程式と価値関数

MDP の目的は、各状態でどのような行動をとるべきかというルール、すなわち方策

$$\pi : \mathcal{S} \rightarrow \mathcal{A} \quad (2.2)$$

を見つけることである。 π を計算するために、状態価値関数

$$V : \mathcal{S} \rightarrow \mathbb{R} \quad (2.3)$$

を導入する。状態価値関数は

$$V(s) \leftarrow \left\langle R(s, a, s') + V(s') \right\rangle_{P(s'|a,s)} \quad (2.4)$$

と計算される。また、 V は、終端状態 $s \in S_f$ に対しては、0 に、それ以外の s に対しては、無限大（実装上は大きい定数）に初期化される。これにより、 V は、ある状態 s で、行動 a をとったときに目的地から s までのコストの期待値となる。

方策のうち V を最小化するものを最適方策 π^* と呼び、 π^* に従ったときの価値関数を最適状態価値関数 $V^*(s)$ とよぶ。 $V^*(s)$ は、ベルマン方程式

$$V^*(s) = \min_{a \in \mathcal{A}} \left\langle R(s, a, s') + V^*(s') \right\rangle_{P(s'|a,s)} \quad (2.5)$$

を満たす。ベルマン方程式は再帰的な構造をしており、ある状態の価値は、そこで最適な行動をとった際に期待される即時報酬と、遷移先の状態の価値の和によって決まることを意味している。

2.2 価値反復アルゴリズムによる経路計画

経路計画を行う際には MDP を有限マルコフ決定過程 (finite MDP) として、計算を行う。状態 $s \in \mathcal{S}$ は、ロボットの位置・向き $xy\theta$ 空間を格子状に離散化して作ったに対し、そこから目的地までのコストを計算した $V(s)$ が計算される。

s を離散化したため、期待値の形式で示したベルマン方程式 (2.5) を総和の形式に変換したものを用いて、反復計算により $V^*(s)$ を求める。詳細な計算とアルゴリズムの手順は以下の通りである。

1. 初期化: 目的地に含まれる状態 $s_f \in \mathcal{S}_f$ の $V(s_f)$ は, 0, その他の s の $V(s)$ は無限大 (実装上は大きい定数) に初期化する.
2. 価値更新: 以下の更新式を, すべての状態 s に対して適用する.

$$\pi(s) \leftarrow \operatorname{argmin}_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s'|a, s) [R(s, a, s') + V(s')] \quad (2.6)$$

$$V_{k+1}(s) \leftarrow \min_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s'|a, s) [R(s, a, s') + V_k(s')] \quad (2.7)$$

ここで k は反復回数を表す.

3. 収束判定: 全状態における価値関数の更新量 $|V_{k+1}(s) - V_k(s)|$ の最大値が, 事前に定めた閾値 ϵ 未満になれば停止する.

また, 一回の反復計算は, 式 (2.6) と式 (2.7) によって行われる. 図 2.2 に s が xy 平面である場合の例を示す. 図 2.2 左では, 環境が格子状に区切られており, 状態空間を構成している. ロボットは, $(x, y)^T = (2, 2)^T$ におり, 行動 a によって矢印の方向に移動したときの状態遷移モデルの分布をグレーで示している. 各状態へ状態遷移する確率は, 状態遷移モデルを各状態で積分したものであり, 枠内に示している. このときの一つの行動 a の式 (2.7) による計算

$$\sum_{s' \in \mathcal{S}} P(s'|a, s) [R(s, a, s') + V_k(s')] \quad (2.8)$$

を図示したものが, 図 2.2 右である. 遷移先の s' に該当する状態の価値 $V(s')$ と移動にかかるコスト $R(s, a, s')$ の和に状態遷移する確率をかけたものの総和を計算している. この計算を xy 空間でそれぞれの行動に対して行い, 最も $V(s)$ が小さくなる行動を選ぶ計算を繰り返すことで, V^* が求まる.

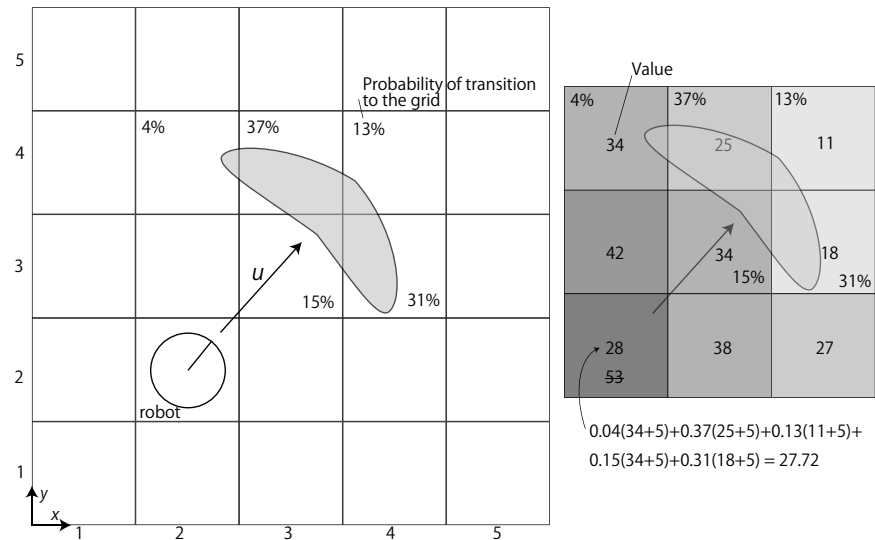


図 2.2 Calculation of the state value function

V は、環境の全域で、式 (2.7) の計算を繰り返すことで正解の値 V^* に近づいていく。図 2.3 は、価値反復 ROS パッケージの V の計算の過程を任意の θ で固定した XY 平面を描画したもので、白から黒くなるほど V の値が大きいことを示している。薄い緑は、値がとても大きい状態、濃い緑は障害物である。このとき、 V の値は目的地の周りから収束していき、目的地から遠いところが最後に収束する。この計算は、「幅優先探索」のコスト計算に相当する処理となる。

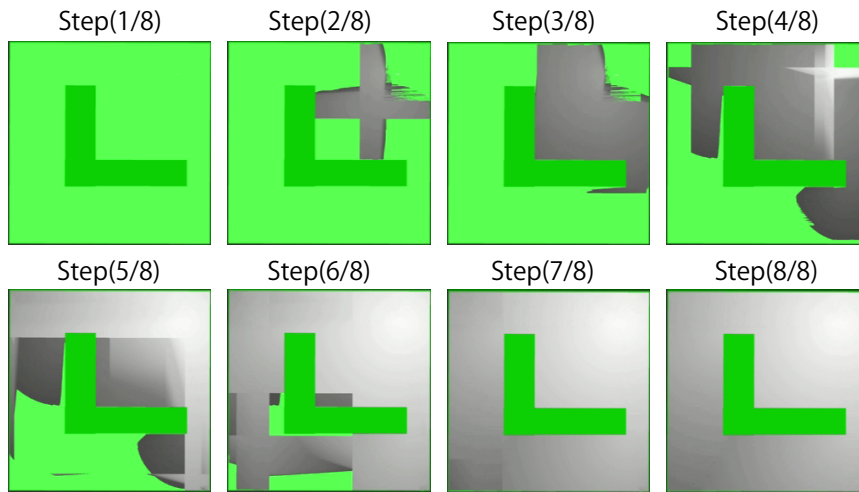


図 2.3 Convergence of the value function

V が収束すると、ロボットは V の値が小さくなるような行動を選択し続けることで、最適な経路を選択して移動できるようになる。これは、環境内のあらゆる場所からゴールへ向かうための最適なポテンシャル関数の勾配が得られることを意味している。価値反復では、人工ポテンシャル法とは違い、局所解の問題が発生しないという強力な数学的保証がある。

また、価値反復を動的な環境で用いるときには、ロボットの周辺で、障害物の情報を追加して計算することで障害物回避が可能になる。この機能は、大域経路計画と独立した機能であり、本研究では、陽には取り扱わない。

また、ロボットが移動を開始するためには、 V^* を厳密に計算する必要はない。 V が完全に収束しなくても、ロボットのいる s から目的地に向かうための適切な勾配が V にできる（目的地に向かって値が単調減少している）と、ロボットは勾配が小さくなる方向へ移動することで目的地に向かうことができる。このような状態を、本稿では「経路が見つかる」と表現する。

しかしながら、この勾配の伝播は A*などの探索手法が経路を探すよりも遅い。価値反復は、探索の手法として見ると幅優先探索になっており、 V の勾配は目的地の周辺からではじめ、それがより遠い地点に伝播していく。またこの伝搬の計算は、ロボットが通らないであろう経路でも行われる。そのため、目的地までの経路が存在すれば、それが複雑に入り組んでいても必ず見つけることはできるが、ロボットは現在地に勾配が到達するま

で、長く待たされることになる。

第 3 章

提案手法

価値反復 ROS パッケージにグラフ探索手法を組み合わせ、グラフ探索手法で算出した経路から V に勾配を作ることで、走り出しまでの時間を短縮する手法を提案する。3.1 節は、文献 [中村 24] で発表した 2 次元を探索する A^* (2D A^*) を組み合わせたものであり、3.2 節は、文献 [中村 25] で発表した 3 次元を探索する A^* (3D A^*) を組み合わせたものである。

3.1 価値反復と 2D A^* を組み合わせた大域計画

目的地を与えられたら、価値反復と並行に 2D A^* で探索を行う。多くの目的地では、2D A^* は、価値反復よりも先に経路を見つけられると考えられる。2D A^* は、 XY 平面を探索するものであり、価値反復が計算を行う $XY\theta$ 空間に比べ、1 つ次元が低い。これにより、特に地図が大きくなったとき、次元の違いから状態（あるいはノード）の数が A^* 探索の方が少なくなると考えられる。そのため、探索にかかる時間が価値反復の勾配の伝播に比べ小さくなることが期待される。

本稿では、 A^* で算出した経路を暫定経路と呼び、暫定経路の情報から、図 3.1 のように、 V の値を書き換える。図のようにロボットが行動を開始する状態から目的地まで、 V の値を少しずつ減らしていくように書き換えることで、価値反復が経路を見つける前に、ロボットを目的地に向かわせるようにする。

V の書き換えは、 A^* の見つけた経路沿いの各 s に対して、

$$V(s) \leftarrow Kf(s) \quad (3.1)$$

という代入の式を用いて行う。 f は s から目的地までの経路上での道のりである。 K は定数であり、 $f(s)$ から $V(s)$ への単位の変換を主に行う。

この手法では、ロボットと目的地の間の環境が単純であり、迷路ようになっていなければ、 A^* で見つかる経路は、価値反復の算出する経路とほぼ一致すると考えられる。このとき、式 (3.1) が与える V の値は、価値反復の計算する V の値との差が小さく、価値反復よりも早く、 V を目的地とロボットとの間で計算できたと考えられる。

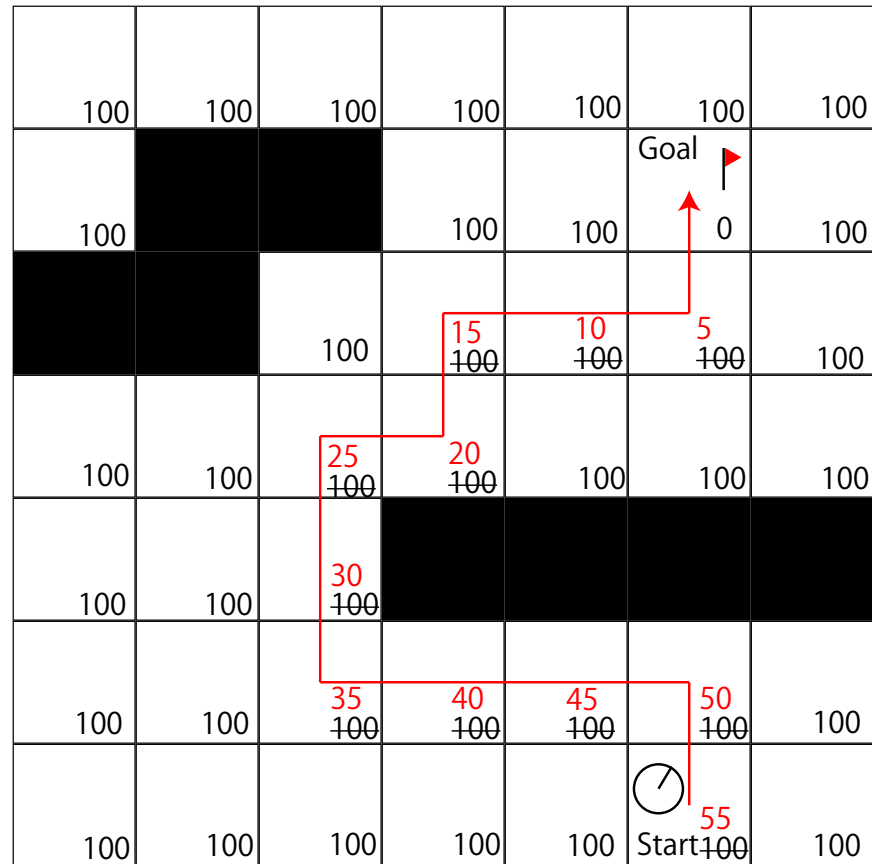


図 3.1 propose method with 2D A*

これにより、価値反復のみの場合よりもロボットが速やかに目的地に向かえるようになる。一方、迷路のような環境だと、A*の経路の算出に時間がかかったり、A*で見つけた経路が遠回りで、そのあとで価値反復がよりよい経路を見つけると、目的地までの時間が増えてしまう可能性がある。

また、A*がほぼ最適な経路を見つけられる場合、価値反復は大域計画に対しては必須ではなくなる。しかし、A*の見つけた経路の最適化や、未知の障害物が現れた場合の V の修正や迂回先の V の計算に必要となる。その一方で、算出する経路が $XY\theta$ において最適ではないものとなり、 V の勾配が思わぬ方向に傾くような悪影響が考えられる。

3.2 価値反復と 3D A*を組み合わせた大域計画

そこで、価値反復と同じ探索空間を持つ A* (3D A*) を用いて経路を算出し、用いる手法を提案する。A*の探索空間でも、価値反復の状態空間と同様にグリッド方向に加え、角度の方向にも離散化し、探索空間を拡張する。

3D A*の算出する経路は、用いるヒューリスティック関数 H が許容的にあるとき、最適性が保証される。これにより、価値反復のコストと大きな差がない暫定的なコストを計

算できることが期待できる。A*は、決定論的な状態遷移を用いて探索を行うのに対し、価値反復は、確率的な状態遷移を行うため、算出される経路は、必ず同じものとはならないが、平面上を探索する 2D A*に比べると価値反復の経路に近くなると考えられる。

その一方で、当然ながら 2D A*に比べて計算量は多くなり、暫定経路の算出にかかる時間は増加すると考えられる。そのため、走り出しまでの時間の短縮の効果は短くなると考えられる。

暫定経路による V の勾配による移動時間の短縮の効果か、3D A*の計算時間の増加量のどちらが多いのか、非自明であり、本稿で検証する。

第 4 章

実装

4.1 A*の実装と価値反復への適用

4.1.1 A*の実装

2D A*の実装

価値反復 ROS パッケージに、A*探索を行うスレッドを 1 つ追加した。ROS 2 のノードの形式で実装し、実装には、`ike_nav`[池邊 23] パッケージ内の `ike_nav_planner` を A*探索のプログラムとして流用した。`ike_nav_planner` のアルゴリズムの部分流用し、通信部分と、ROS 2 のノードとして使うための部分を書き換えた。

`ike_nav_planner` はマップと現在地と目的地に対して A*で計算された経路を出力する。使われる A*のヒューリスティック関数 H_{2D} は、ユークリッド距離であり、

$$H_{2D}(s) \triangleq W \sqrt{(x_c - x_g)^2 + (y_c - y_g)^2} \quad (4.1)$$

と定義する。ここで、 (x_c, y_c) は各離散状態の中心の座標、 (x_g, y_g) は目的地の中心地点の xy 座標、 W は、定数である。この W を調整し真のコストを H_{2D} が上回らないようにすることで、経路の最適性が保証される。

3D A*の実装

そこで、 $XY\theta$ 空間で探索を行う A*を実装した。A*の具体的な実装については、`ike_nav`[池邊 23] パッケージ内の A*探索ノード (`ike_nav_planner`) を 3 次元での探索に拡張し、価値反復 ROS パッケージに移植して利用した。3D A*は、価値反復と同様に、グリッドに加えて θ 方向にも離散化し、グラフを構築する。 XY 方向の遷移コストと θ 方向の遷移コストは単位が一致するよう距離から時間に変更した。

ヒューリスティック関数 H_{3D} については、 H_2 から、角度の情報を付け加えた。

$$H_{3D}(s) \triangleq W_1 \sqrt{(x_c - x_g)^2 + (y_c - y_g)^2} + W_2 |\theta_c - \theta_g| \quad (4.2)$$

に変更する。ここで、 θ_c は各離散状態の中心の角度、 $\theta_c - \theta_g$ は、 (x_c, y_c, θ_c) から見た

(x_g, y_g) の方角である． W_1, W_2 は定数である． W_1 と W_2 の比は，ロボットが一定時間あたりに移動できる距離と方向転換できる角度の比で決まる．

4.1.2 価値反復と A* の併用

価値反復 ROS パッケージ内に A* で算出した経路上で，式 (3.1) にしたがって， V の値を書き換えるコードを追加した．これにより V の勾配がロボット付近に生成され，ロボットは走り出すことができると考えられる．

2D A* の暫定経路の適用

2D A* が出力する経路 p_i ($i \in \mathbb{N}$) は XY 平面上のものであり， θ 方向の情報がない．つまり， $p_i = (x_i \ y_i)^T$ と表せ，式 (3.1) 上の f は，この経路の集合 $p_i \in P$ に対して実数を返す関数と解釈できる．よって，式 (3.1) を $s_{p_i} \in \mathcal{S}$ を用いて

$$V(s_{p_i}) \leftarrow Kf(p_i) \quad (4.3)$$

と実装した．ここで， s_{p_i} は， θ 全体の集合 $\Theta \subset \mathcal{S}$ を用いて

$$\forall \theta \in \Theta, s_{p_i} = \begin{pmatrix} x_i \\ y_i \\ \theta \end{pmatrix} \quad (4.4)$$

に従う．つまり XY 平面上で同じ位置にある離散状態にはすべて同じ値を代入する． θ 方向の V の値は，すべて同じであるが，価値反復の計算によって勾配が形成される．この実装では，価値反復の生成する勾配とは異なる値を代入してしまっている．これにより，通常起き得ない V の値の遷移によって移動に対して悪影響がでると考えられる．

3D A* の暫定経路の適用

3D A* が出力する経路 p_i は， $XY\theta$ 空間上のものであるため， $p_i = (x_i \ y_i \ \theta_i)^T$ と表せる．このとき f の引数と V の引数が一致する．そのため，

$$V(p_i) \leftarrow Kf(p_i) \quad (4.5)$$

と実装した．

第 5 章

シミュレータ実験

提案手法の走り出しまでの時間を短縮する効果の有無を検証するために、シミュレータを用いて実験を行う。実験を行うシミュレータ環境は、動的な障害物が存在せず、障害物回避による移動時間の増加を考慮する必要がない大域計画による移動時間を計測する理想的な環境である。

5.1 実験条件

シミュレータ実験では、千葉工業大学津田沼キャンパスで得られた図 5.1 の地図を使用する。紙面横方向が 300m、縦方向が 200m の 6ha の環境の地図で、形式は、ROS のナビゲーションスタックで用いられる占有格子地図である。白色が障害物のない画素、黒色が障害物のある画素、灰色が不明な画素を表す。この地図のパラメータは表 5.1 の通りである。

価値反復 ROS パッケージでは、地図のグリッドに加え、 θ 方向にも離散化を行い S を構成する。 S および価値反復 ROS パッケージの諸元を表 5.3 に、行動のリストを表 5.2 に示す。状態の数は 1 億に達し、移動可能なエリアだけでも 1 千万に達する。

実験で使用するシミュレータは、図 5.2 から作成したシミュレータを使用する。シミュレータには、ROS の標準的なシミュレータである Gazebo[Koenig 04] を使用し、シミュレータ内で用いる静的な障害物の作成には、占有格子地図から作成するプログラム[池邊 24]を使用した。

シミュレータ内で走行させるロボットは、図 5.3 の差動二輪型のロボットである。差動

表 5.1 Configurations of the Map

map size	294.6[m]×199.95[m]
cell resolution	0.15[m]×0.15[m]
number of cells	2,615,346
number of free cells	165,076
the area of the free cells	3714.98[m ²]

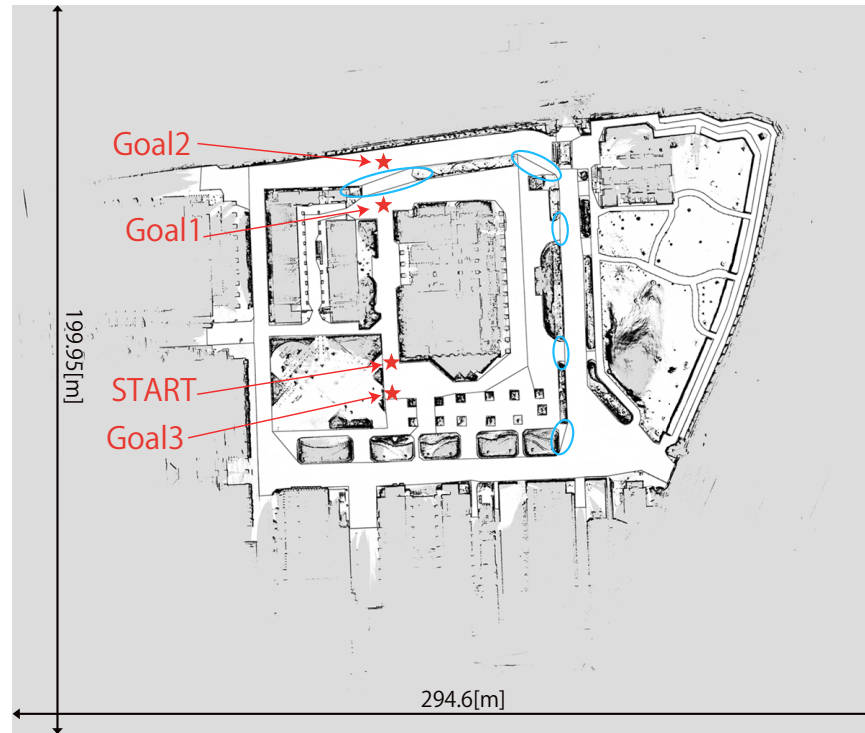


図 5.1 a map for navigation

表 5.2 List of Action in Value Iteration

Type	Linear Velocity [m/s]	Angular Velocity [deg/s]
forward	0.3	0.0
back	-0.2	0.0
right	0.0	-20.0
right forward	0.2	-20.0
left	0.0	20.0
left forward	0.2	20.0

表 5.3 Parameters of Value Iteration

θ -axis status resolution	6 deg
number of states	156,920,760
number of free states	9,904,560
number of actions	6
Δt	1

二輪型のロボットは，進行方向に対して前後に進むことができるが，横に進むことができない．そのため，当然ながらゴールの方向へ進むにはその方向かその反対を向く必要がある．つまり，行動には向きの制限がつくため，位置と方向の3次元で経路を探索することで移動の効率化が計れる．

実験に用いた計算機は，CPU として Ryzen 9 7945HX3D (16 コア 32 スレッド)，

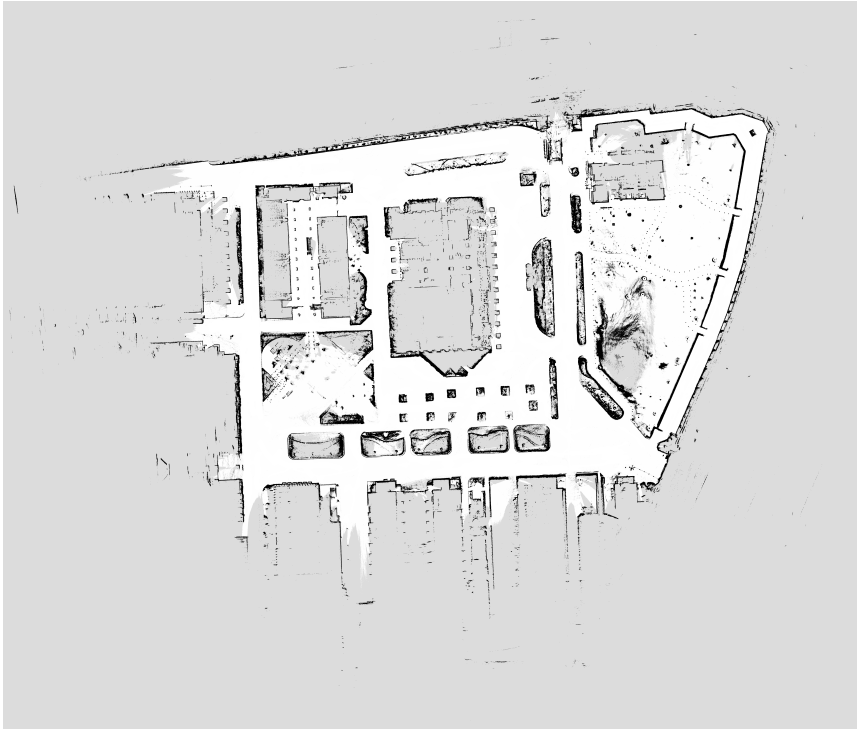


図 5.2 a map for localization

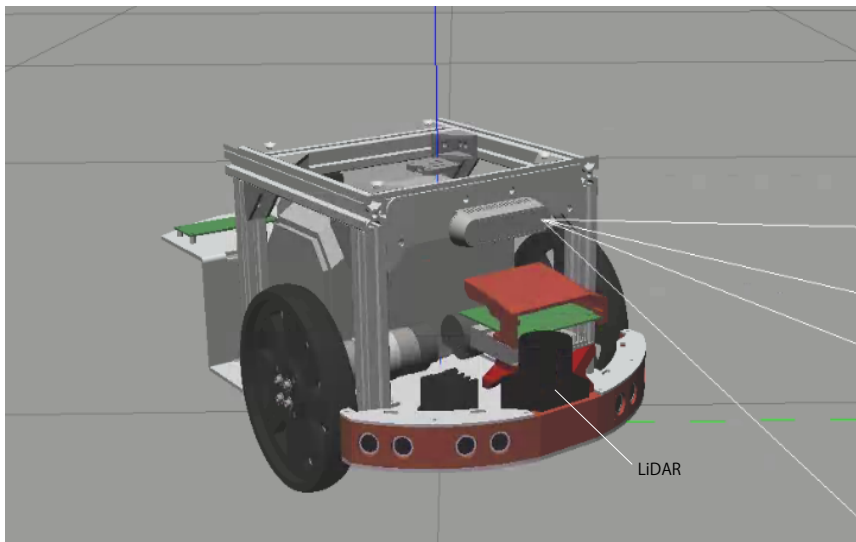


図 5.3 a robot for simulation

DRAM として DDR5-4800 32GB が 2 枚の合計 64GB 搭載されたものである。価値反復 ROS パッケージは、CPU ですべて計算するため、コアの数が重要である。このコンピュータは、現代の家庭用コンピュータの中でも、コアの数の多い CPU を搭載している。DRAM については、地図の大きさに応じて決定した。

価値反復 ROS パッケージは、計算の効率化のため並行に V を計算できるよう実装され

ている。環境の全域で V を計算する大域計画器と、ロボットの周囲で V を計算する局所計画器に任意のスレッドの数を割り当てることができる。本稿の実験では、大域計画器に 8、局所計画器に 6 のスレッドを割り当てた。また、 A^* は別のプロセスで動作し、CPU から見ると、 A^* は 1 つのスレッドである。また、シミュレータの Gazebo にも計算機のリソースを使用している。

実験では、図 5.1 に示す 3 つの目的地で比較を行った。Goal1 は、式 (4.1) あるいは式 (4.2) のヒューリスティック関数を使ったときにすぐ経路が見つかるように設定した。同様に、Goal2 は、障害物に阻まれ、大きな迂回が必要なように、Goal3 は、旋回の所要時間を考慮しない式 (4.1) の A^* が不利になるように、スタートでのロボットの向きと反対方向に設定した。Goal2 に迂回が必要となるのは、図 5.1 右の丸で囲ったところやその他数カ所に、移動ロボットが跨げない段差が存在するからである。

比較するプログラムは、価値反復 ROS パッケージ単体 (VI)、2D A^* と価値反復 ROS パッケージを併用したもの (VI+2D A^*)、3D A^* と価値反復 ROS パッケージを併用したもの (VI+3D A^*) の 3 つである。これとは別に、2D A^* の経路を pure pursuit (付録 A) で追従したもので移動時間を計測し、ロボットが移動するのにかかる時間を計測する。

5.2 実験結果

VI, VI+2D A^* , VI+3D A^* で 5 回ずつ試行した結果を表 5.4 に示す。また、2D A^* と 3D A^* が、各目的地まで暫定経路の算出にかけた時間を表 5.5 に、Goal1, 2 までの 2D A^* の経路を追従した移動時間を表 5.6 に示す。

Goal1 では、VI+2D A^* と VI+3D A^* のどちらの移動時間も、VI の移動時間に比べ、減少しており、移動時間短縮の効果を確認できる。また、表 5.5 から、 A^* の計算時間と移動時間を比較して移動時間の 1.5% 以内の時間で、 A^* の計算が終了していることが分かる。

Goal3 では、設置の意図通り、VI+3D A^* が総移動時間の最も小さくなっており、 A^* を 3D にする効果が確認できている。計算時間は、VI+2D A^* に比べ大きいですが、総移動時間は、小さくなっているのは、VI+2D A^* の勾配の作り方が、角度方向は VI によって作られているため、その計算に時間がかかったためと考えられる。

Goal2 については、VI+2D A^* と VI+3D A^* のどちらも VI より大幅に時間がかかった。 A^* は最短の経路を見つけていたが、価値反復の計算の途中で V の勾配に分岐が生じ、ロボットが迷走するという現象が見られた。

各手法の Goal2 までロボットが移動するのにかかった時間を 2D A^* の経路を追従するのにかかる時間と仮定したとき、VI の経路が見つかるまでの計算時間と、VI+2D A^* と VI+3D A^* の迷走した時間を計算したものを表 5.7 に示す。VI の計算時間は、総移動時間から移動時間を引いたもの、VI+2D A^* と VI+3D A^* の迷走した時間は、総移動時間から A^* の計算時間と移動時間を引いたものである。

表 5.7 から VI と VI+2D A^* の計算時間は、おおよそ同じくらいで、VI+3D A^* の計算

時間は、それよりも大きいことが分かる。VI+2D A*は、迷走による総移動時間の増加という問題を解決すれば、計算に時間がかかるような目的地においても移動時間の増加は小さいと考えられる。

上記のロボットが迷走する問題の対策については、A*探索を定期的に何度も実行する方法が考えられる。そうすることで V に分岐が生じた際、その状態からゴールまでの別の経路を A*が発見できる可能性がある。ただし、価値反復が計算した正確な V を上書きして変更する可能性もあるため、式 (3.1) での勾配のつけかたに工夫が必要とされる。

表 5.4 Result with the Map of Outdoor Environment

methods	traveling time (standard deviation)[s]		
	Goal1	Goal2	Goal3
VI	226 (7)	597 (1)	62 (7)
VI+2D A*	214 (1)	1135 (129)	69 (2)
VI+3D A*	219 (1)	1414 (169)	54 (4)

表 5.5 Calculation Time for Temporal Path

methods	calculation time (standard deviation)[s]		
	Goal1	Goal2	Goal3
2D A*	2.3 (0.3)	27 (0.1)	2.4 (0.1)
3D A*	2.5 (0.2)	189 (8.5)	2.8 (0.1)

表 5.6 Travering Time for Temporal Path

methods	travering time (standard deviation)[s]	
	Goal1	Goal2
2D A*	204 (0)	571 (1)

表 5.7 Time for Goal 2

Methods	times for Goal 2 (standard deviation)[s]			
	All	Calculate	Moving	Astraying
VI	597 (1)	26 (2)	571 (1)	
2D A*	1135 (129)	27 (0.1)	571 (1)	537 (130)
3D A*	1414 (169)	189 (8.5)	571 (1)	654 (179)

第 6 章

結論

本稿では、従来の価値反復 ROS パッケージの走り出すまでにかかる時間を A* を併用することで短縮する手法を提案した。実装する A* について、 xy 平面を探索する A* と、 $xy\theta$ 空間を探索する A* の 2 種類の A* を実装し、その効果をシミュレータを用いて検証した。その結果、A* の計算時間が十分に小さかった場合、走り出しまでの時間が短縮されることが分かった。その一方で、A* の計算に時間がかかる場合には、状態価値関数に分岐が生成され、ロボットの移動に時間がかかってしまう現象を確認した。

$xy\theta$ を探索する A* は、実験のうち有利となるように設定された目的地においては、 xy 平面を探索する A* よりも移動時間を短縮することができていた。その一方で、単純な移動経路となるような目的地や、計算に時間のかかる目的地においては、 xy 平面を探索する A* よりも移動時間を短縮することができず、移動時間の短縮の効果よりも、A* 探索の計算時間の増加の影響のほうが大きいと考えられる。そのため、 $xy\theta$ を探索する A* よりも xy 平面を探索する A* のほうが走り出しまでの時間を短縮するための探索手法としては適していた。

本稿では、指定された 1 つの目的地へ向かうタスクによって、走り出すまでの時間の減少の効果を測定したが、1.1 節で述べたように、指定された経路を追従するタスクも考えられる。価値反復でこのタスクに取り組む場合、その指定された経路の終点を目的地としたとき指定された経路が最適なものであるならば、1 つの目的地へ向かうタスクと同じように行うことができる。

その一方で、経路同士の交差があるなど、最適でないならば、複数の目的地を置き、順に向かうことで達成できる。複数の目的地の置き方については、価値反復が大域経路計画の情報を用いて局所経路計画を行う点や、大域経路計画の計算コストが大きい点からなるべく少ない数置くことが望ましい。最低限の目的地を置いた場合でも、恣意的に目的地を設定できるため、目的地間の経路を単純なものにでき、A* の計算時間を小さくできる。つまり、指定された経路を追従するタスクの場合においても、価値反復と A* を併用する手法を用いることで移動時間の短縮できると考えられる。

今後は、より環境の大域的な構造を捉え、かつ計算時間の小さい、つまり、計算量の小さくなるような手法を用いて走り出すまでの時間の短縮を試みる。例えば、複数の

解像度で A*探索を行う方法や，作成した一般ボロノイ図をトポロジカルマップとして探索を行う方法がある．

他にも，走り出すまでの時間を短縮するには，探索手法を用いずに，マップを通路を除いてマスクし，部分的に価値反復を行う方法が考えられる．部分的な状態価値関数の勾配を生成する方法として，探索手法を用いるのではなく，価値反復を使うことで，齟齬が起きることを防ぐことができると考えられる．

付録 A

pure pursuit による経路追従

A.1 概要

pure pursuit は，グラフ探索手法などで求めた経路を追従するアルゴリズムの 1 つである．ロボットが現在の位置から経路上の目標点（look ahead point）へ向かって，一定の曲率を持つ円弧軌道を描くように制御することで経路を追従する．実装が簡単であるため，本研究の実験で用いた．

A.2 導出

まず，本研究で使用する差動駆動方式のロボットの運動モデルを考える．世界座標系 Σ_G におけるロボットの位置および姿勢を $p = [x, y, \theta]^T$ とする．ロボットの制御入力として並進速度 v および旋回角速度 ω を与えた場合，その運動方程式は

$$\dot{p} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix} \quad (\text{A.1})$$

と記述される．

次にロボットの座標系 Σ_r における目標点の座標を求める．世界座標系 Σ_G におけるロボットの旋回中心（左右車輪軸の中点）の座標 $p_r = [x, y, \theta]^T$ から経路上の前方距離 L （look ahead distance）離れた点を目標点 $p_{target} = [t_x, t_y]^T$ とする． Σ_r をロボットの進行方向を x 軸として取ると， Σ_r における目標点 $p' = [x', y']^T$ は，回転行列を用いて

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} t_x - x \\ t_y - y \end{bmatrix} \quad (\text{A.2})$$

のように求められる．

次に，ロボットが目標点に到達するために必要な p_r と p_{target} 間をつなぐ円弧の曲率 κ を求める．この円弧は，図 A.1 に示すように， p_r における接線方向とロボットの向きが一致するように取り付ける．円弧の半径を R とすると， κ は

$$\kappa = \frac{1}{R} \quad (\text{A.3})$$

と表せ、ロボットの向きと p_r から見た p_{target} の向きの差を a とすると、

$$\sin a = \frac{y'}{L} \quad (\text{A.4})$$

の関係式が成り立つ．式 (A.4) を用いて R は

$$R = \frac{\frac{L}{2}}{\sin a} = \frac{L^2}{2y'} \quad (\text{A.5})$$

である．よって κ は、式 (A.3) と式 (A.5) から

$$\kappa = \frac{2y'}{L^2} \quad (\text{A.6})$$

となる．

最後に、追従に必要な目標角速度 ω_{cmd} を求める．差動 2 輪ロボットにおいて、旋回角速度は並進速度と曲率の積で与えられる．したがって、並進速度 v として式 (A.6) より、 ω_{cmd} は

$$\omega_{cmd} = v \frac{2y'}{L^2} \quad (\text{A.7})$$

と計算できる．ここで、 y' はロボットから見た目標点の横偏差を表しており、式 (A.7) は「横偏差に比例して旋回角速度を与える」という P 制御的な特性を持つことがわかる．

A.3 実装

実装では、予め経路と並進速度が与えられ、一定周期でロボットの位置に対して、目標点を探査し、式 (A.7) を計算する．疑似コードを図 A.3 に示す．実装上、必ずしもロボットと経路の距離が L より小さいとは限らないため、最小の L (L_{min}) を設定している．

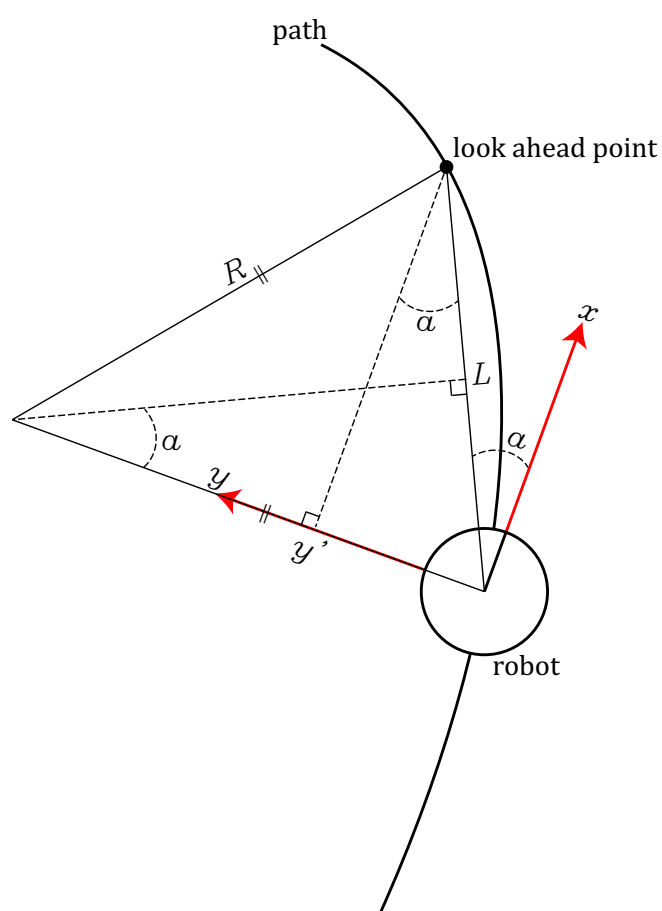


図 A.1 how follow the path by pure pursuit

Algorithm 1 Pure Pursuit Algorithm for Differential Drive Robot

Require: Current pose $p_t = (x_t, y_t, \theta_t)$, Target path $\mathcal{P}$, Current velocity v_t
Ensure: Angular velocity command ω_t
Parameters:
 L_{min} : Minimum look-ahead distance

 k : Look-ahead gain

 $L_d \leftarrow \max(k \cdot v_t, L_{min})$
 $\triangleright$ Update look-ahead distance

 $g \leftarrow \text{SEARCHTARGET}(\mathcal{P}, p_t, L_d)$
 $\triangleright$ Find look-ahead point (g_x, g_y)
if g is not found **then**
return 0

 $\triangleright$ Stop if no target

end if
 $\alpha \leftarrow \text{atan2}(g_y - y_t, g_x - x_t) - \theta_t$
 $\triangleright$ Angle to target

 $\alpha \leftarrow \text{NORMALIZEANGLE}(\alpha)$
 $\triangleright$ Normalize to $[-\pi, \pi)$
 $\omega_t \leftarrow v_t \cdot \frac{2 \sin \alpha}{L_d}$
 $\triangleright$ Calculate angular velocity

return ω_t

図 A.2 demo code of pure pursuit

参考文献

- [Alverhed 24] E. Alverhed, S. Hellgren, H. Isaksson and L. Olsson, H. Palmqvist, and J. Flodén. Autonomous last-mile delivery robots: A literature review. *European Transport Research Review*, Vol. 16, No. 4, 2024.
- [Arkin 90] Ronald C. Arkin. Integrating Behavioral Perceptual World Knowledge in Reactive Navigation. No. 6, pp. 105–122, 1990.
- [Bellman 57] Richard Bellman. *Dynamic Programming*. Dover Publications, 1957.
- [Engel 14] J. Engel, T. Schöps, and D. Cremers. LSD-SLAM: Large-scale direct monocular SLAM. In *European Conference on Computer Vision (ECCV)*, pp. 834–849, 2014.
- [E.W. 59] Dijkstra E.W. A note on two problems in connexion with graphs. *Numerische Mathematik*, Vol. 1, pp. 269–271, 1959.
- [Fox 99] Dieter Fox, Wolfram Burgard, Frank Dellaert, and Sebastian Thrun. Monte Carlo Localization: Efficient Position Estimation for Mobile Robots. In *Proc. of AAAI-99*, pp. 343–349, 1999.
- [Fragapane 21] Giuseppe Fragapane, René de Koster, Fabio Sgarbossa, and Jan Ola Strandhagen. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *European Journal of Operational Research*, Vol. 294, No. 2, pp. 405–426, 2021.
- [Gerkey 09] Brian Gerkey. gmapping, 2009. Accessed on 27.11.2025.
- [Hart 68] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, Vol. 4, No. 2, pp. 100–107, 1968.
- [Karaman 11] Sertac Karaman and Emilio Frazzoli. Incremental sampling-based algorithms for optimal motion planning. In *Robotics: Science and Systems VI*. The MIT Press, 08 2011.
- [Khatib 85] O. Khatib. Real-time obstacle avoidance for manipulators and mobile robots. In *Proceedings. 1985 IEEE International Conference on Robotics and Automation*, Vol. 2, pp. 500–505, 1985.

- [Koditschek 90] Daniel E Koditschek and Elon Rimon. Robot navigation functions on manifolds with boundary. *Advances in Applied Mathematics*, Vol. 11, No. 4, pp. 412–442, 1990.
- [Koenig 04] Nathan Koenig and Andrew Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2149–2154, Sendai, Japan, 2004.
- [Kohler 16] Damon Kohler, Wolfgang Hess, and Holger Rapp. Cartographer, 2016. Accessed on 27.11.2025.
- [Koide 24] Kenji Koide, Shuji Oishi, Masashi Yokozuka, and Atsuhiko Banno. GLIM: 3D Range-Inertial Localization and Mapping with GPU-Accelerated Scan Matching Factors. *Robotics and Autonomous Systems*, Vol. 179, p. 104750, 2024.
- [Konolige 00] K. Konolige. A gradient method for realtime robot control. In *Proceedings. 2000 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000) (Cat. No.00CH37113)*, Vol. 1, pp. 639–646 vol.1, 2000.
- [LaValle 98] Steven M. LaValle. Rapidly-exploring random trees : a new tool for path planning. *The annual research report*, 1998.
- [LLC 18] Open Navigation LLC. Navigation2, 2018. Accessed on 1.12.2025.
- [MA 15] Raúl Mur-Artal, J. M. M. Montiel, and Juan D. Tardós. ORB-SLAM: A Versatile and Accurate Monocular SLAM System. *IEEE Transactions on Robotics*, Vol. 31, No. 5, pp. 1147–1163, 2015.
- [Macenski 22] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, Vol. 7, No. 66, p. eabm6074, 2022.
- [Moosmann 11] Frank Moosmann and Christoph Stiller. Velodyne slam. In *2011 IEEE Intelligent Vehicles Symposium (IV)*, pp. 393–398, 2011.
- [Thrun 05] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic ROBOTICS*. MIT Press, 2005. (邦訳) 上田訳: 確率ロボティクス, 毎日コミュニケーションズ (2007) .
- [Ueda 23] Ryuichi Ueda, Leon Tonouchi, Tatsuhiko Ikebe, and Yasuo Hayashibara. Implementation of brute-force value iteration for mobile robot path planning and obstacle bypassing. *Journal of Robotics and Mechatronics*, Vol. 35, No. 6, pp. 1489–1502, 2023.
- [Zhang 14] Ji Zhang and Sanjiv Singh. Loam: Lidar odometry and mapping in real-time. In *Robotics: Science and System Conference*, Vol. 2, pp.

- 1-9, 2014.
- [横山 19] 横山和寿. ヤンマーテクニカルレビュー 自動運転農機「ROBOT TRACTOR」の紹介. https://www.yanmar.com/jp/about/technology/technical_review/2019/0403_1.html, 2019. (2025-11-27 参照).
- [楽天 25] 楽天グループ株式会社. 楽天、商品配送サービス「楽天無人配送」において、新たなロボットの導入、対象店舗や地域の拡大などサービスを拡充, 2025. https://corp.rakuten.co.jp/news/press/2025/0226_01.html (2025-12-11 参照).
- [株式 17] 株式会社クボタ. 自動運転農機「アグリロボトラクタ」を市場”初”投入. <https://www.kubota.co.jp/news/2017/17-23j.html>, 2017. (2025-11-27 参照).
- [原 25] 原祥亮, 萬礼応, 富沢哲雄, 伊達央, 大川一也, 大矢晃久. つくばチャレンジ 2024 全チームの技術動向調査. ロボティクス・メカトロニクス講演会, pp. 2A2-R09, 2025.
- [厚生 24] 厚生労働省. 第 9 期介護保険事業計画に基づく介護人材の必要数について. 2024.
- [国土 24] 国土交通省. 令和 5 年度 宅配便等取扱個数の調査及び集計結果. 2024.
- [国立 23] 国立社会保障・人口問題研究所. 日本の将来推計人口（令和 5 年推計）. 2023.
- [上田 19] 上田隆一. 詳解 確率ロボティクス Python による基礎アルゴリズムの実装. 講談社, 2019.
- [上田 21] 上田隆一. value_iteration: real-time value iteration planner package for ROS, 2021. Accessed on 1.12.2025.
- [上田 24] 上田隆一. value_iteration2: value iteration for ROS 2, 2024. Accessed on 27.11.2025.
- [前山 97] 前山祥一, 大矢晃久, 油田信一. 移動ロボットの屋外ナビゲーションのための オドメトリとジャイロのセンサ融合によるデッドレコニング・システム. Vol. 15, No. 8, pp. 1180-1187, 1997.
- [池邊 23] 池邊龍宏. ike_nav: Simple ROS 2 Navigation Stack with C++ Implementation, 2023. Accessed on 27.11.2025.
- [池邊 24] 池邊龍宏. raspicat_map2gazebo, 2024. Accessed on 25.12.2025.
- [中居 22] 中居達. ベイズ学習とマルコフ決定過程. コロナ社, 〒112-0011 東京都文京区千石 4-46-10, 2022.
- [中村 24] 中村啓太郎, 登内りオン, 永木悠暉, 林原靖男, 上田隆一. 自律移動ロボットのための価値反復ベースの大域計画器における A*探索による暫定経路の算出. 第 25 回システムインテグレーション部門講演会, pp. 1131-1133, 2024.
- [中村 25] 中村啓太郎, 林原靖男, 上田隆一. 自律移動ロボットのための価値反復と

- A*探索を組み合わせた大域経路計画. 第 43 回日本ロボット学会学術講演会, pp. 3Q3–06, 2025.
- [塚越 16] 塚越貴哉, 明比建, 北村光教, 鈴木太郎, 天野嘉春. オープンソース GNSS ライブラリを用いた遊歩道環境での 自律移動ロボットナビゲーションのための位置推定. Vol. 52, No. 5, pp. 276–283, 2016.
- [内閣 16] 内閣府. 第 5 期科学技術基本計画. 2016.
- [日本 22] 日本国政府. 道路交通法の一部を改正する法律. 令和四年法律第三十二号, 2022. https://elaws.e-gov.go.jp/law/335AC0000000105/20230401_504AC0000000032?hit_toc=Sp%255B7%255D%252CSp%255B18%255D%252CSp%255B18%255D%252CSp%255B18%255D%252CSp%255B18%255D%252CSp%255B29%255D%252CSp%255B29%255D%252CSp%255B29%255D%252CSp%255B29%255D%252CSp%255B31%255D%252CSp%255B31%255D%252CSp%255B39%255D%252CSp%255B39%255D%252CSp%255B44%255D%252CSp%255B44%255D%252CSp%255B66%255D%252CSp%255B66%255D (2025-11-27 参照).
- [日本 24] 日本国政府. 農業の生産性の向上のためのスマート農業技術の活用の促進に関する法律. 令和六年法律第六十三号, 2024. <https://elaws.e-gov.go.jp/law/506AC0000000063> (2025-11-26 参照).
- [友納 20] 友納正裕, 原祥堯. SLAM の現状と今度の展望. Vol. 64, No. 2, pp. 45–50, 2020.

業績

- 中村啓太郎, 登内リオン, 永木悠暉, 林原靖男, 上田隆一
「自律移動ロボットのための価値反復ベースの大域計画器における
A*探索による暫定経路の算出」
第 25 回自動計測制御学会
システムインテグレーション部門講演会 2024 講演論文集,
pp.1131-1133, 2024.
- 中村啓太郎, 林原靖男, 上田隆一
「自律移動ロボットのための価値反復と A*探索を組み合わせた大域経路計画
— 探索空間を 3 次元に拡張した A*との組み合わせの検討—」
第 43 回日本ロボット学会学術講演会講演論文集,
3Q3-06, 2025.